LeDron-James Documentation & FAQ

Saif Edine Safi, Filippo Marcatili, Anas Soussane, Riccardo Leone

December 9, 2024

Contents

1 Introduction **1**

2 Architecture Overview **2**

3 Core Data Structures **2**
- 3.1 Drone . . . 2
- 3.2 Cache . . . 2
- 3.3 Enums . . . 3

4 Core Functionalities **3**
- 4.1 Execution Loop . . . 3
- 4.2 Crash Handling . . . 3
- 4.3 Packet Processing . . . 3
- 4.4 Flooding Requests . . . 3
- 4.5 Packet Utilities . . . 4

5 Customization Options **4**
- 5.1 Terminal Logging . . . 4
- 5.2 Simulation Controller ACKs . . . 4

6 FAQ **4**

7 Conclusion **5**

1 Introduction

LeDron-James represents the pinnacle of drone node implementations for WG_2024. Congratulations on acquiring this masterpiece of efficient and reliable code. Designed to communicate seamlessly through channels, process

packets in adherence to specified protocols, and integrate with a Simulation Controller, LeDron-James offers a robust and elegant solution. This document provides a comprehensive overview of its architecture, functionalities, and customization options.

2 Architecture Overview

The `Drone` structure serves as a node in a simulated network with the following responsibilities:

- Listening for commands from the Simulation Controller.
- Receiving and forwarding packets from neighboring drones.
- Applying source-routing logic to determine packet paths.
- Handling flooding requests, acknowledgments (ACK), negative acknowledgments (NACK), and responses.
- Managing a crashed state that affects drone behavior.

3 Core Data Structures

3.1 Drone

- `id: NodeId` – Unique identifier of the drone.
- `pdr: f32` – Packet Delivery Ratio, the probability of packet forwarding success.
- `packet_send: HashMap<NodeId, Sender<Packet»` – Maps neighbor IDs to sending channels.
- `packet_recv: Receiver<Packet>` – Receives incoming packets.
- `controller_send: Sender<DroneEvent>` – Sends events to the Simulation Controller.
- `controller_recv: Receiver<DroneCommand>` – Receives commands from the Simulation Controller.
- `cache: Cache` – A struct that holds the flood request history and the crash state.

3.2 Cache

- `history_floodreq: HashMap<NodeId, u64>` – Records flood requests by initiator and flood ID.

- `crashed: bool` – Indicates if the drone is in a crashed state.

3.3 Enums

- `SendingCodes`: Represents outcomes for packet sending.
- `RoutingCodes`: Represents routing outcomes like `Correct`, `DestinationArrived`, and `HopsMismatch`.

4 Core Functionalities

4.1 Execution Loop

The drone prioritizes commands from the Simulation Controller using `select_biased!`. Commands are handled by `handle_command`, while packets are processed by `drone_behaviour`.

4.2 Crash Handling

If the drone crashes, the Simulation Controller sends `RemoveSender` commands to neighbors, severing communication. The drone only processes residual `Ack`, `Nack`, and `FloodResponse` packets while discarding others.

4.3 Packet Processing

Packets are processed based on routing information:

- Valid packets are forwarded, potentially applying PDR for `MsgFragment`.
- If the drone is the final destination:
  - `FloodRequest`: Handled by `handle_flooding_req`.
  - Other types: Generate `DestinationIsDrone` NACKs.
- If `HopsMismatch` occurs, it might still be a `FloodRequest`. After checking for that, if it's not a `FloodRequest`, appropriate NACKs are generated.

4.4 Flooding Requests

Flood requests are managed to prevent redundancy:

- If a flood request is recognized, a `FloodResponse` is sent.
- If new, it is forwarded to neighbors after recording the request.

4.5 Packet Utilities

- `build_packet_flood_response`: Creates a response to a flood request.
- `build_packet_nack`: Creates a NACK packet for errors like dropped or invalid routing.

5 Customization Options

5.1 Terminal Logging

Terminal logging is a highly useful feature for debugging, providing detailed insights into the drone's processing of information and packets at each step. This makes it not only engaging to observe the drone's operations but also significantly easier to identify and resolve any issues. For those who prefer a quieter execution, we can provide a version of LeDron-James without terminal logging upon request.

5.2 Simulation Controller ACKs

By default, the drone does not send `PacketSent` events for every packet. This can be enabled for detailed event tracking.

6 FAQ

Q: What happens if the drone crashes? A: Routes involving the drone are invalidated, and neighbors sever communication. Residual ACK/NACK-/FloodResponse packets are processed; others are dropped.

Q: How are corrupted Source Routing Headers handled? A: The drone sends an `UnexpectedRecipient` NACK to the Simulation Controller, allowing corrective action.

Q: How are unknown packet types handled? A: They are logged and either dropped or NACKed based on the drone's state.

Q: Does the drone have a contact method? A: Yes! Join our Discord channel for updates and to contact our amazing staff: `discord.gg/Bd6YT9K3y3`.

7 Conclusion

LeDron-James is a versatile, reliable drone node implementation tailored for the WG_2024 simulation environment. Its debugging options, structured error handling, and responsive support team make it an ideal choice for various simulation scenarios.